Real-Time Wildfire Risk Prediction System using a Hybrid Batch–Streaming Big Data Pipeline on Hadoop and Apache Spark

Amitha Mahesh
Artificial Intelligence and Datascience
Amrita Vishwa Vidyapeetham
Ettimadai, Coimbatore

Modiyam Meghana Reddy
Artificial Intelligence and Datascience
Amrita Vishwa Vidyapeetham
Ettimadai, Coimbatore

Nitheka A
Artificial Intelligence and Datascience
Amrita Vishwa Vidyapeetham
Ettimadai, Coimbatore

Shivvanni S
Artificial Intelligence and Datascience
Amrita Vishwa Vidyapeetham
Ettimadai, Coimbatore

Dr. Sreeja B P
Artificial Intelligence and Datascience
Amrita Vishwa Vidyapeetham
Ettimadai, Coimbatore

***Abstract*—Wildfire warning systems that rely on a single data source tend to raise either too many false alarms (weather-only rules) or too many unreliable alerts (social-media-only detection), and real data feeds are frequently incomplete. This paper presents a hybrid, Lambda-style big data pipeline that addresses these problems using the Hadoop and Spark ecosystem. The batch layer stores 22.1 GB of NOAA Integrated Surface Database observations (53,174 compressed files) in a Hadoop Distributed File System (HDFS) cluster with one NameNode, three DataNodes and replication factor three, and uses Apache Spark with Scala to parse, clean and model the data with Spark MLlib. The speed layer uses Spark Structured Streaming to fill gaps in incomplete weather readings with per-station last-known values, to recover missing geotags from post text, and to raise an alert only when risky weather and a fire-related social post agree in the same spatial grid cell within a $\pm$2-minute window. Verified alerts are served through a Flask API to a live Leaflet map dashboard. On a 1% sample of the cleaned data, Decision Tree and Random Forest classifiers reach an F1 score of 0.9974 compared with 0.9807 for Logistic Regression, and a full-scale Random Forest run on all 22.1 GB reaches an F1 score of 0.9975. Because the fire label is derived from a physical threshold rule rather than recorded fire events, these scores show that the pipeline learns the rule at scale and are not a claim of real-world fire prediction; we discuss this and other limitations explicitly.**

***Index Terms*—big data analytics, wildfire early warning, Apache Spark, Scala, HDFS, Structured Streaming, Lambda architecture, social sensing, missing data**

I. Introduction

Wildfires cause loss of life, property damage and ecological harm, and the conditions that favour their ignition and spread, namely high temperature, low humidity and strong wind, are well understood. Early warning therefore depends on how quickly and how reliably such conditions can be detected and confirmed on the ground. Traditional warning rules are driven by weather observations alone. A weather-only rule fires on every hot and windy day, so most of its alerts are not followed by an actual fire, which erodes the trust of responders. Social media offers an independent, near-instant signal because residents often report smoke or flames before official detection, but such posts are noisy, may be unrelated rumours, and only a small fraction carry a geotag [5], [6].

The underlying data are also genuinely *big*. Weather archives accumulate hourly observations from thousands of stations over many years, and live feeds produce a continuous stream of weather readings and posts in heterogeneous formats. This combination of large volume, high velocity, mixed formats and uncertain quality matches the classic "V" characteristics of big data and motivates a distributed, fault-tolerant design rather than a single-machine script.

This paper describes the design, implementation and evaluation of a hybrid big data pipeline for wildfire risk prediction. The system stores a 22.1 GB historical weather archive in a replicated HDFS cluster, uses Spark and Scala to clean the data and train and compare machine learning models at scale, and uses Spark Structured Streaming to verify live alerts by cross-checking two independent evidence streams. Our contributions are as follows.

1) An end-to-end Lambda-style architecture that combines a 22.1 GB batch layer and a live speed layer in a single Scala code base on Spark 3.5.0, backed by a Dockerised HDFS cluster with replication factor three.
2) A *dual-evidence verification* mechanism: an alert is produced only when risky weather and a fire-related social post occur in the same grid cell within a $\pm$2-minute event-time window, implemented as a stream–stream join with watermarks.
3) A *missing-data-resilient* streaming design that fills missing weather fields with per-station last-known values and recovers missing coordinates from the city name in post text, with every alert carrying `gap_filled` and `geo_inferred` flags for explainability.
4) A systematic model comparison and hyper-parameter

study (Logistic Regression, Decision Tree and nine Random Forest settings) with an honest discussion of what the scores do and do not demonstrate.
5) A live dashboard (Flask API and Leaflet map) that makes the verified alerts and model statistics visible to a non-technical user.

The rest of the paper is organised as follows. Section II reviews related work. Section III formulates the problem and describes the dataset. Section IV presents the architecture, and Section V the implementation. Section VI describes the experimental setup, Section VII reports and discusses the results, Section VIII states the limitations, and Section IX concludes.

II. Related Work

A. Machine Learning for Wildfire Prediction

Early data-mining work predicted the burned area of forest fires from meteorological variables using regression models and support vector machines [1]. Subsequent research applied a wide range of machine learning methods to fire occurrence, spread and susceptibility mapping, as summarised in a comprehensive review [2]. Ensemble tree methods such as Random Forests [3] are popular in this area because they handle non-linear interactions between weather variables and need little tuning. Most of this literature is evaluated offline on a fixed dataset of modest size, and it stops at prediction without a live verification or alerting layer.

B. Social Sensing for Emergencies

Social media users can act as "social sensors". Sakaki et al. showed that Twitter posts can be used to detect an earthquake in near real time [5], and a survey of mass-emergency social media processing describes the typical difficulties of such data, namely noise, duplication, missing location and the need for credibility assessment [6]. These works motivate using posts as supporting evidence, but they also show that posts alone are unreliable. Our design therefore treats a post as *confirming* evidence for a weather condition and not as a standalone alarm.

C. Big Data Frameworks

MapReduce introduced a simple programming model for large-scale batch processing on commodity clusters [7], and HDFS provides the replicated, fault-tolerant storage layer underneath [8]. Spark generalised the model with resilient distributed datasets (RDDs) that support in-memory reuse and lineage-based recovery [9], added the DataFrame and Spark SQL abstraction with a query optimiser [10], and added MLlib for distributed machine learning [12]. Structured Streaming extends the same DataFrame API to unbounded data with event-time processing, watermarks and exactly-once output semantics [11], which draws on the dataflow model for out-of-order data [13]. The Lambda architecture combines a batch layer for accuracy over history with a speed layer for low-latency views [14]. Our system follows this separation.

D. Missing Data

Real sensor feeds contain missing values. Statistical treatments such as imputation are standard [15]. For a streaming setting where latency matters, simple carry-forward (last-observation) imputation is cheap and can be applied per key without a global pass. We adopt this approach per station and make every imputed alert explicit through a quality flag rather than hiding it.

E. Research Gap

Based on the above, existing wildfire studies are largely batch-only, do not combine independent weather and crowd evidence in one verification step, do not explicitly handle incomplete live feeds, and rarely demonstrate the pipeline at a scale that requires distributed storage and processing. Our work addresses these four points together.

III. Problem Formulation and Dataset

A. Problem Statement

Given (i) a large archive of historical weather observations and (ii) two live streams, one of weather readings and one of social posts, build a distributed pipeline that learns weather-based fire risk at scale and issues a *verified* real-time alert for a location only when both streams provide supporting evidence, while tolerating missing fields.

B. Big Data Characteristics

The project exhibits the following characteristics.

- **Volume:** 22.1 GB of compressed data in 53,174 files.
- **Velocity:** two continuous streams processed as micro-batches (weather every 5 s and posts every 3 s in our generators).
- **Variety:** fixed-width text records, JSON weather readings and free-text posts.
- **Veracity:** sentinel values for missing measurements, missing fields in live data, and unverified social posts.

C. Historical Dataset

The batch layer uses the NOAA Integrated Surface Database (ISD) [4], a public archive of surface weather observations. We downloaded the 2020–2023 station-year files with `lftp`, which resulted in 53,174 gzip-compressed files with a total size of 22.1 GB (average file size about 0.42 MB) and loaded them into HDFS. Each line is one observation in a fixed-width format. Table I lists the fields used. Missing wind speed and temperature are encoded by the sentinels `9999` and `+9999`, respectively.

D. Risk Label

The ISD archive contains weather only and no record of fires, so there is no ground-truth fire label. We therefore define a proxy label from the widely used physical intuition that hot and windy conditions are dangerous:

$$y = \begin{cases} 1, & T > 28^{\circ}\mathrm{C} \ \wedge \ W > 5\ \mathrm{m/s} \\ 0, & \text{otherwise,} \end{cases} \tag{1}$$

TABLE I
ISD FIELDS USED (FIXED-WIDTH LAYOUT)

Attribute	Columns	Scaling
Station identifier (USAF)	5–10	–
Date (YYYYMMDD)	16–23	–
Latitude	29–34	÷1000
Longitude	35–41	÷1000
Wind speed (m/s)	66–69	÷10
Air temperature (°C)	88–92	÷10
Dew point (°C)	94–98	÷10

where T is air temperature and W is wind speed. The consequences of this choice for the interpretation of the results are discussed in Section VII and Section VIII. The same criterion is used by the speed layer to decide whether a weather reading is "risky".

E. Live Data

Because real-time feeds are not freely available for a student deployment, two Python generators simulate them and write JSON files to HDFS landing folders. The weather generator produces temperature, wind speed and humidity for a fixed set of stations (Los Angeles, San Francisco and Las Vegas in our configuration), alternating between normal and spike regimes (spike: temperature 28–42 °C, wind 6–15 m/s, humidity 5–20%; normal: 10–27 °C, 0–5 m/s, 30–70%). To imitate real feeds, each reading randomly drops fields: all sensors present in 50% of readings, only temperature in 18%, only wind in 18% and neither in 14%. The social generator produces genuine fire-related posts and unrelated posts with a position scattered around a station, and 30% of posts have no geotag but name the city in the text.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the architecture. It has a batch layer for history and a speed layer for live data, both written in Scala on Spark and sharing HDFS as the storage substrate.

A. Storage: HDFS Cluster

HDFS stores large files as fixed-size blocks that are replicated across DataNodes, while the NameNode keeps the file-system metadata [8]. Our cluster runs in Docker Compose on WSL2 (Ubuntu) with one NameNode, three DataNodes, a YARN ResourceManager, a NodeManager and a HistoryServer. The replication factor is three, so each block is stored on all three DataNodes, which tolerates the loss of up to two DataNodes without data loss. The archive resides under `/bda_project/raw/weather/`, and the streaming landing zones are `/bda_project/incoming/weather`, `.../social` and `.../weather_filled`.

B. Processing: Apache Spark

Spark executes a job as a directed acyclic graph (DAG) of stages split at shuffle boundaries. Transformations are lazy and are executed only when an action is invoked [9]. Narrow transformations such as `filter` and `map` are pipelined inside one stage, while wide transformations such as aggregations and tree-building shuffle data and create new stages. We use the DataFrame/Spark SQL API [10] for relational operations, the RDD-level `flatMap` for record parsing, Spark MLlib [12] for learning, and Structured Streaming [11] for the speed layer.

C. Deployment Mode

Jobs are launched with `spark-submit --master local[*]` in client deploy mode inside the official `apache/spark:3.5.0` container attached to the Hadoop Docker network so that `hdfs://namenode:9000` resolves. In this mode the driver and the executor share one JVM and tasks run as threads, one per CPU core, so the Spark UI shows a single executor named *driver*. HDFS is therefore genuinely distributed while Spark itself is single-node, a limitation we discuss in Section VIII. The same application jar can be submitted to YARN by changing the master setting.

V. IMPLEMENTATION

A. Batch Layer: Parsing and Cleaning

The raw lines are read from HDFS as text. Gzip is not a splittable format, so Spark creates one partition per file, which gives high parallelism here because of the large number of files (53,174). Each line is parsed with a `flatMap` wrapped in `Try`: the fixed-width substrings for wind and temperature are extracted, sentinel values become `None`, values are scaled, and any malformed line is dropped. The option `spark.sql.files.ignoreCorruptFiles` lets the job skip truncated archives instead of failing. Rows missing temperature or wind are then filtered out, and the label of Eq. (1) is added. Temperature and wind speed are assembled into a feature vector with `VectorAssembler`.

Processing 22.1 GB on a single machine required three runtime adjustments: the driver memory was raised from the default of about 434 MB to 6 GB, shuffle partitioning was customised, and the spill directory was redirected to a larger volume.

B. Batch Layer: Models, Caching and Tuning

The cleaned data are split 80/20 into training and test sets with a fixed seed (42). We evaluate three algorithms from Spark MLlib: Logistic Regression (maximum 50 iterations), a Decision Tree (depth 5) and a Random Forest [3]. The Random Forest is tuned over a grid of `numTrees` $\in \{20, 50, 100\}$ and `maxDepth` $\in \{3, 5, 10\}$, giving nine settings. Because eleven models are fitted, repeatedly re-reading and re-parsing 22.1 GB of gzip data would be wasteful. The comparison job therefore draws a 1% random sample (seed 42), splits it 80/20, and calls `persist(MEMORY_AND_DISK)` on both parts so that all models reuse the in-memory data. A separate training job fits a 50-tree Random Forest on the entire cleaned data set.

Performance is measured with Spark's `MulticlassClassificationEvaluator` using the F1 score, weighted precision and weighted recall:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}, \quad (2)$$

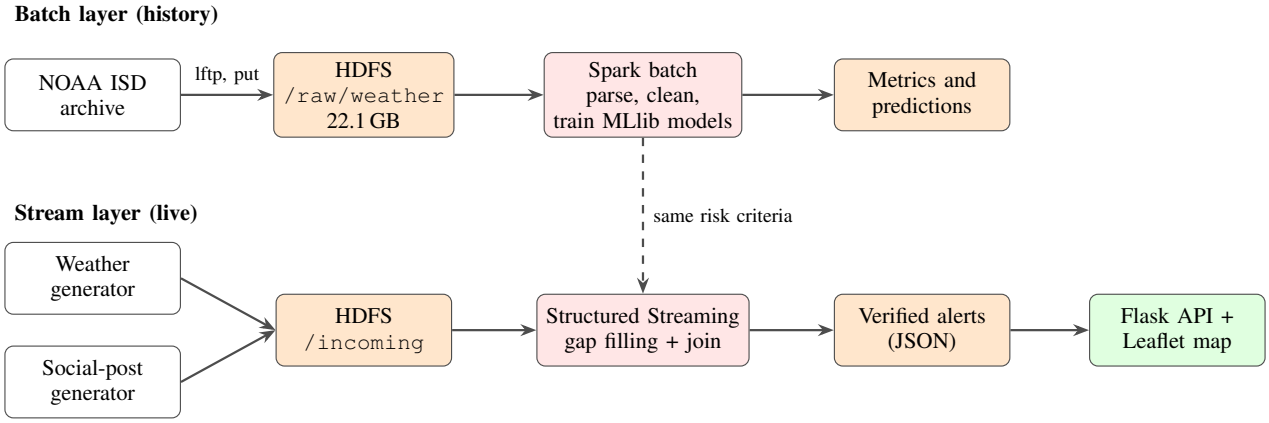


Fig. 1. Lambda-style architecture. The batch layer processes the 22.1 GB archive stored in HDFS; the speed layer verifies live alerts and feeds the dashboard.

Algorithm 1 Per-station gap filling (one micro-batch)

Require: batch of readings B; state S mapping station $\rightarrow (t, w, h)$

1: **for all** reading $r \in B$ ordered by event time **do**
2: $(t', w', h') \leftarrow S[r.\text{station}]$ (or null if unseen)
3: $t^* \leftarrow r.t$ if present, else t'; similarly w^*, h^*
4: gaps $\leftarrow$ fields that were absent in r but filled
5: $S[r.\text{station}] \leftarrow (t^*, w^*, h^*)$
6: emit $(r, t^*, w^*, h^*, \text{gaps})$ to `weather_filled`
7: **end for**

where the weighted versions average the per-class values by class support.

C. Speed Layer: Handling Missing Data

The weather stream may contain readings in which temperature, wind or both are absent. A `foreachBatch` sink applies per-station last-known-value imputation (Algorithm 1) and writes the filled readings, with a flag that lists which fields were imputed, to `/incoming/weather_filled`. If a field has never been seen for a station, it stays null, and a null can never satisfy the risk condition, so an unseen value cannot cause an alert. For social posts, a missing latitude/longitude is recovered from the city name that appears in the text and the post is flagged `geo_inferred`.

D. Speed Layer: Verification by Stream–Stream Join

Two streaming sources are read with a declared schema from HDFS: the filled weather and the social posts. A weather reading is *risky* if it satisfies Eq. (1), and a post is a candidate if it is classified fire-related by a simple keyword filter. Because two streams can only be joined with an equality key, each coordinate is quantised to a 1° grid cell (about 100 km):

$$g(\text{lat}, \text{lon}) = \big(\text{round}(\text{lat}),\ \text{round}(\text{lon})\big). \tag{3}$$

A verified alert is produced when a candidate post and a risky reading share a grid cell and their event times differ by at most two minutes:

$$g_s = g_w \ \wedge\ |t_s - t_w| \le 2 \text{ min}. \tag{4}$$

TABLE II
EXPERIMENTAL ENVIRONMENT

Component	Configuration
Storage	HDFS: 1 NameNode, 3 DataNodes, replication 3
Cluster extras	YARN ResourceManager, NodeManager, HistoryServer
Processing	Apache Spark 3.5.0, Scala 2.12, Java 11, sbt
ML library	Spark MLlib (`spark.ml`)
Streaming	Spark Structured Streaming (micro-batch)
Spark mode	`local[*]`, client mode, driver memory 6 GB
Runtime	Docker, Docker Compose, WSL2 Ubuntu
Serving	Flask API, Leaflet.js
Data	22.1 GB NOAA ISD, 53,174 `.gz` files

Watermarks bound the join state so that old records are discarded, and a checkpoint directory makes the query restartable. The grid size was chosen after observing that a 0.1° cell was too small: posts scattered around a station often fell into a neighbouring cell and could never match. The 1° cell fits each of the three stations and keeps them far apart, so alerts from different cities cannot mix. Each alert row carries the station, location, temperature, wind speed, the post text and the two quality flags `gap_filled` and `geo_inferred`.

E. Dashboard

Alerts are written as JSON files and read by a Flask server that exposes `/api/alerts` and `/api/model`. The web page uses Leaflet.js, hosted locally so that it does not depend on an external script host, with a dark base map. Each station is a pin that turns red and pulses while it has an active alert, a zone circle is drawn around it, and the pin returns to green after three minutes without a new alert. The page also shows the model statistics and a table of recent alerts.

VI. EXPERIMENTAL SETUP

Table II lists the software and cluster configuration. All experiments ran on a single laptop under WSL2.

Batch experiments. The full-data job trains a 50-tree Random Forest on the entire cleaned archive using the 80/20 split. The comparison and tuning job runs on a 1% sample for the reasons given above. Metrics are computed on the held-out

TABLE III
MODEL COMPARISON ON A 1% SAMPLE (TEST SPLIT)

Model	F1	Prec.	Recall	Time (s)
Logistic Regression	0.9807	0.9799	0.9826	33.4
Decision Tree (depth 5)	0.9974	0.9976	0.9973	25.9
Random Forest (20 trees, depth 3)	0.9974	0.9976	0.9973	21.1

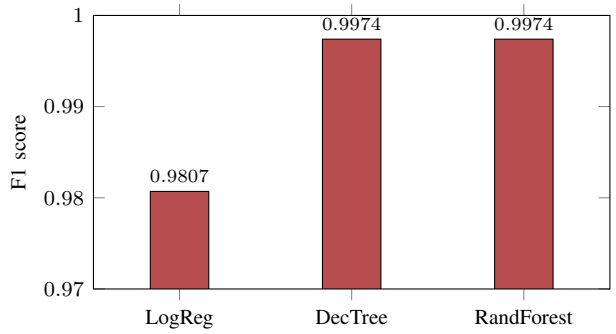


Fig. 2. F1 score of the three algorithms on the 1% sample. The vertical axis is zoomed to 0.97–1.00 so that the difference is visible.

TABLE IV
RANDOM FOREST TRAINING TIME (S) FOR THE TUNING GRID (F1 = 0.9974 FOR ALL SETTINGS)

Trees	depth 3	depth 5	depth 10
20	21.1	27.0	48.4
50	31.1	53.0	93.1
100	53.7	91.4	161.9

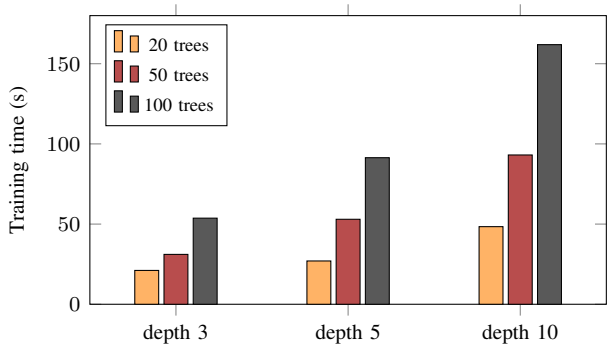


Fig. 3. Random Forest training time over the tuning grid. Accuracy is identical for all nine settings.

20% test portion, and wall-clock training time is recorded for every model.

Streaming experiments. The streaming job, the two generators and the dashboard run concurrently. We validate the behaviour functionally, that is, that gaps are filled, geotags are recovered, the join emits alerts only for co-located and co-timed evidence, and the dashboard reflects them. The Spark UI (port 4040) is used to monitor jobs, stages, executors and streaming input/processing rates.

VII. RESULTS AND DISCUSSION

A. Full-Scale Training

The 50-tree Random Forest trained on the full 22.1 GB cleaned archive achieved an F1 score of 0.9975, weighted precision of 0.9977 and weighted recall of 0.9974 on the held-out test split. This confirms that the pipeline processes the complete data set end to end on a single machine after the memory adjustments described above.

B. Model Comparison and Tuning

Table III and Fig. 2 compare the algorithms on the 1% sample. Logistic Regression reaches an F1 score of 0.9807, while the Decision Tree and the Random Forest reach 0.9974. All nine Random Forest settings give identical scores (F1 0.9974, precision 0.9976, recall 0.9973) to the fourth decimal.

Table IV and Fig. 3 show the training time of the nine Random Forest settings. Since the F1 score does not change, the additional trees and depth only add cost: the largest setting (100 trees, depth 10) takes 161.9 s, about 7.7 times the 21.1 s of the smallest (20 trees, depth 3). The best configuration is therefore the cheapest one that reaches the top score, namely 20 trees with depth 3.

C. Interpretation

The results are explained by the way the label is built. Eq. (1) is a conjunction of two thresholds, which a single decision tree of small depth can represent exactly, whereas Logistic Regression is a linear model and approximates the rectangular decision region less well. This is why every tree-based model saturates at the same score and why tuning brings no accuracy gain. It also means that the score reflects the model's ability to *reproduce the rule at scale*, not to forecast real fires. We therefore report these numbers as evidence that the distributed pipeline is correct and scalable, and we do not claim real-world predictive accuracy.

D. Speed Layer Behaviour

The streaming job behaves as designed. Readings with missing fields are completed from the station's last known values and are marked with the `gap_filled` flag. A reading for which a field has never been observed cannot trigger an alert. Posts without geotags are located from the city name and flagged `geo_inferred`. A verified alert appears only when a risky reading and a fire-related post fall in the same grid cell within two minutes, which removes alerts that would be raised by weather alone and ignores posts with no weather support. The effect of the grid size was visible in practice: changing the cell from 0.1° to 1° made matches frequent enough for a live demonstration. The dashboard then shows the pulsing pin, the zone circle and the alert table in near real time.

E. Discussion of Design Choices

We considered duel evidence method because a single source is either too eager (weather) or too unreliable (posts). Requiring both is a simple precision-oriented rule that is easy

to explain to responders. We took 1% sample for tuning because eleven fits on 22.1 GB with a single-machine Spark would be impractical, and the full-data run confirms that the sample result carries over to the full data (0.9974 on the sample against 0.9975 on the archive).

VIII. Limitations

We state the limitations of this work explicitly.

1) **Proxy label.** The fire label is a threshold rule and not a recorded fire event, so the F1 scores measure rule reproduction. Validation against real fire records is required before any claim of predictive value.
2) **Single-node Spark.** Spark runs in `local[*]` mode, so the cluster-level behaviour of Spark (shuffle across machines, executor failure) is not exercised, although HDFS is truly distributed.
3) **Simulated live data.** The weather and social streams are generated and not collected from live sources, so the alert behaviour is a functional demonstration and not a measured field result.
4) **Stream rule versus model.** The speed layer applies the same threshold criteria as the batch label and does not load the saved Random Forest.
5) **Simple verification.** Fire-related posts are identified with a keyword filter and the grid cell is coarse (about 100 km).
6) **In-memory imputation state.** The last-known values are held in the driver and are lost if the streaming job restarts.
7) **Weighted metrics.** Weighted precision and recall are averaged by class support, which can hide poor performance on a rare positive class.

IX. Conclusion and Future Work

We presented a hybrid batch–streaming big data pipeline for wildfire early warning built on HDFS, Spark, Scala and Structured Streaming. The batch layer processed 22.1 GB of NOAA observations in 53,174 files and trained a Random Forest with an F1 score of 0.9975. A comparison on a sample showed that tree-based models outperform Logistic Regression (0.9974 versus 0.9807) and that tuning only increases cost. The speed layer verifies alerts using two independent evidence streams, tolerates missing data and explains its output with quality flags, and a live map dashboard presents the result. The system demonstrates distributed storage, large-scale batch processing and event-time stream joins in one reproducible code base, while the limitations above bound the claims that can be made.

Future work includes validating the pipeline against real fire records such as NASA FIRMS active-fire detections, adding humidity, vegetation and terrain features, loading the trained model directly in the streaming job, replacing the keyword filter with a text classifier on a real social feed, persisting the imputation state with Spark's stateful operators, and running the application on a multi-node YARN or Kubernetes cluster.

Acknowledgment

We thank their out faculty guide Dr. Sreeja B P and the department for their guidance and support during this work.

References

[1] P. Cortez and A. Morais, "A data mining approach to predict forest fires using meteorological data," in *Proc. 13th Portuguese Conf. Artificial Intelligence (EPIA)*, 2007, pp. 512–523.

[2] P. Jain, S. C. P. Coogan, S. G. Subramanian, M. Crowley, S. Taylor, and M. D. Flannigan, "A review of machine learning applications in wildfire science and management," *Environmental Reviews*, vol. 28, no. 4, pp. 478–505, 2020.

[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] A. Smith, N. Lott, and R. Vose, "The Integrated Surface Database: Recent developments and partnership enhancements," *Bull. Amer. Meteor. Soc.*, vol. 92, no. 6, pp. 704–708, 2011.

[5] T. Sakaki, M. Okazaki, and Y. Matsuo, "Earthquake shakes Twitter users: Real-time event detection by social sensors," in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 851–860.

[6] M. Imran, C. Castillo, F. Diaz, and S. Vieweg, "Processing social media messages in mass emergency: A survey," *ACM Computing Surveys*, vol. 47, no. 4, pp. 67:1–67:38, 2015.

[7] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.

[8] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[9] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28.

[10] M. Armbrust *et al.*, "Spark SQL: Relational data processing in Spark," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2015, pp. 1383–1394.

[11] M. Armbrust *et al.*, "Structured Streaming: A declarative API for real-time applications in Apache Spark," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2018, pp. 601–613.

[12] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *J. Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[13] T. Akidau *et al.*, "The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015.

[14] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Shelter Island, NY, USA: Manning, 2015.

[15] R. J. A. Little and D. B. Rubin, *Statistical Analysis with Missing Data*, 2nd ed. Hoboken, NJ, USA: Wiley, 2002.